STAR EAST

Ensuring Big Data Quality: Practical Testing Strategies

April 30, 2025

9:45am–10:45am

With

Kunal Jain

Agenda

1. A real pipeline failure
2. Why data testing is hard
3. Testing the pipeline
4. Streaming & Kafka
5. Schema drift & Contracts
6. CI/CD Quality Gates

A real pipeline failure

A real pipeline failure

App code

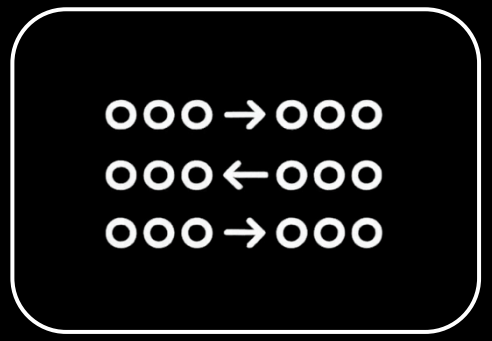
- Lint
- Security
- Unit tests
- Integration tests

Data Pipeline

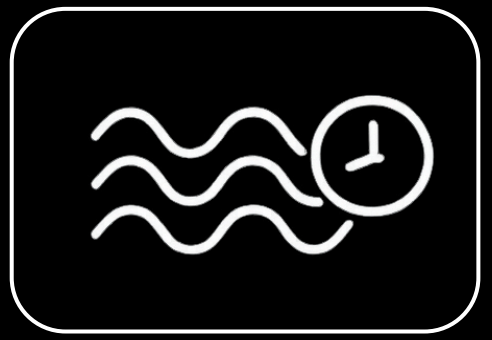
?

Why data testing is hard

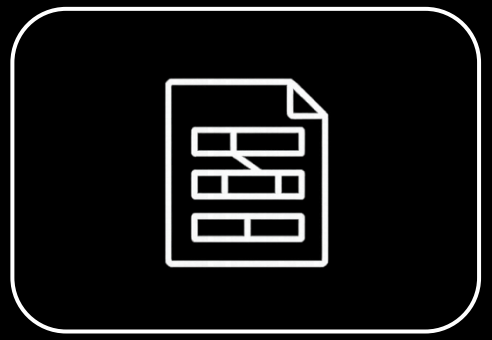
Scale



Non-determinism



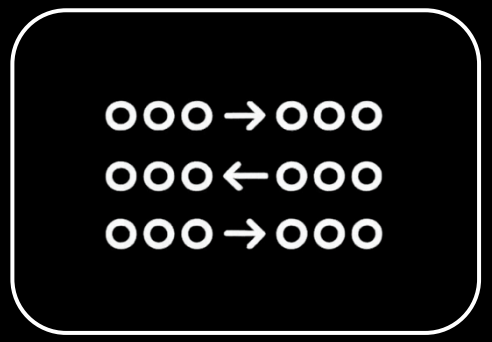
Time & State



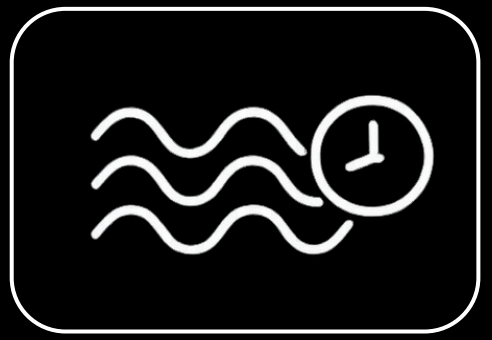
Schema Drift

Why data testing is hard

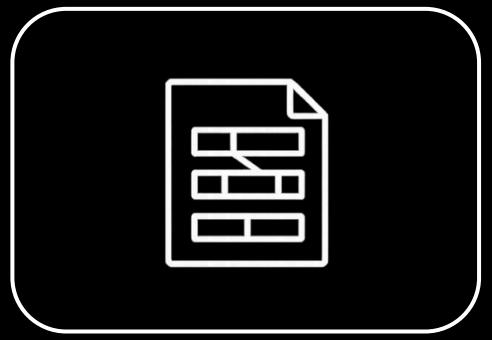
Scale



Non-determinism



Time & State



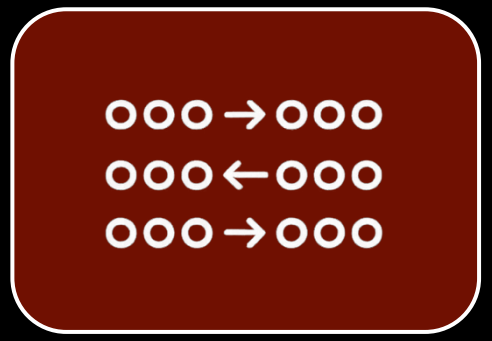
Schema Drift

500M rows.

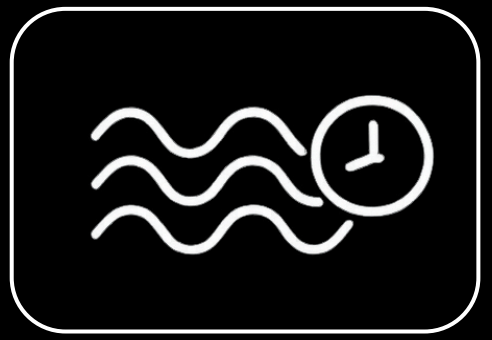
You can't assertEquals.

Why data testing is hard

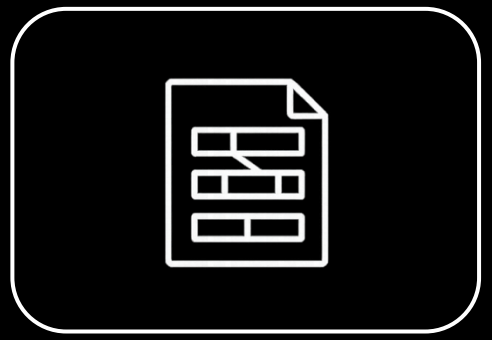
Scale



Non-determinism



Time & State

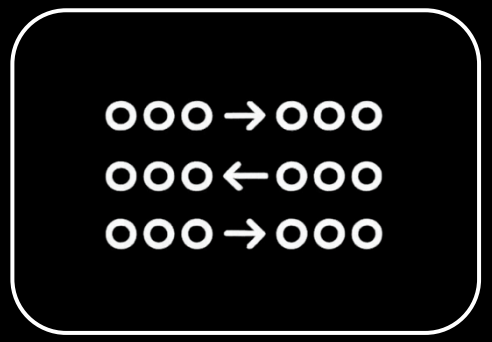


Schema Drift

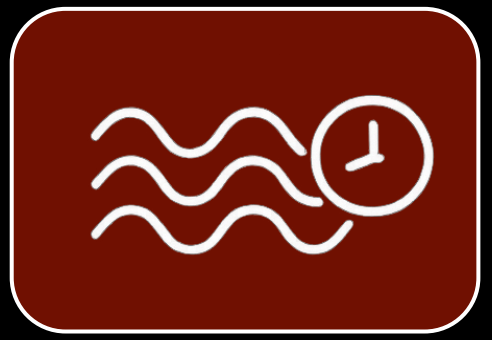
Spark doesn't guarantee row order.

Why data testing is hard

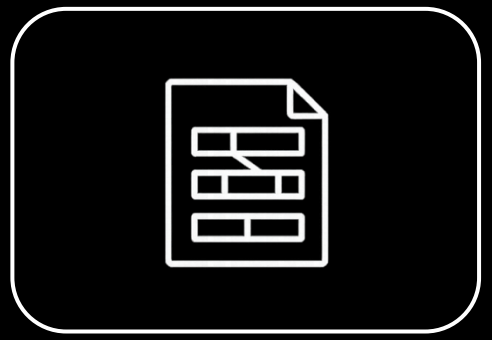
Scale



Non-determinism



Time & State

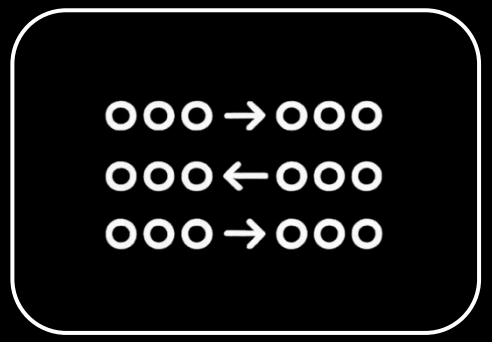


Schema Drift

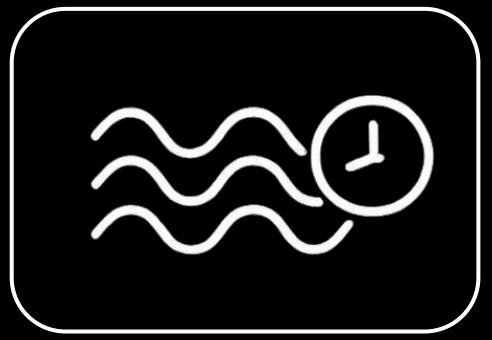
Streaming is a process, not a function.

Why data testing is hard

Scale



Non-determinism



Time & State

Schema Drift

Upstream changes without warning.

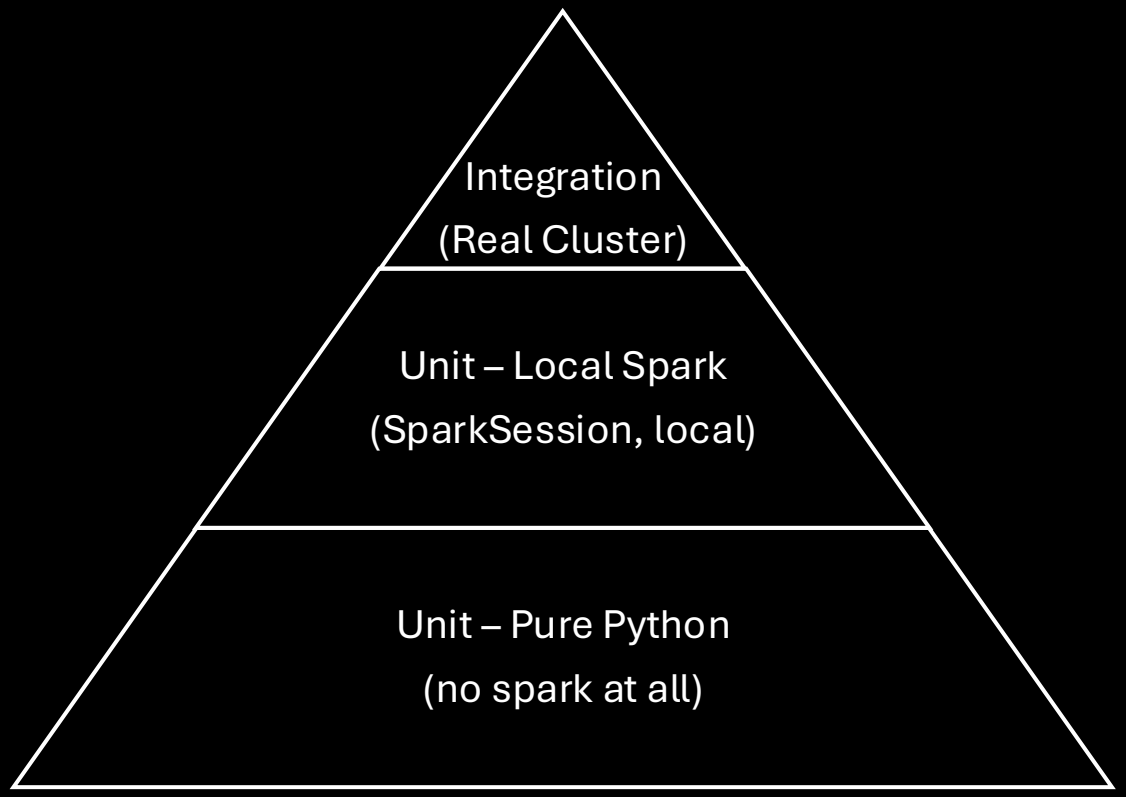
Why Data Testing Is Different

15-20%

of revenue lost to poor data quality

- Forbes Tech Council

The Spark Testing Pyramid



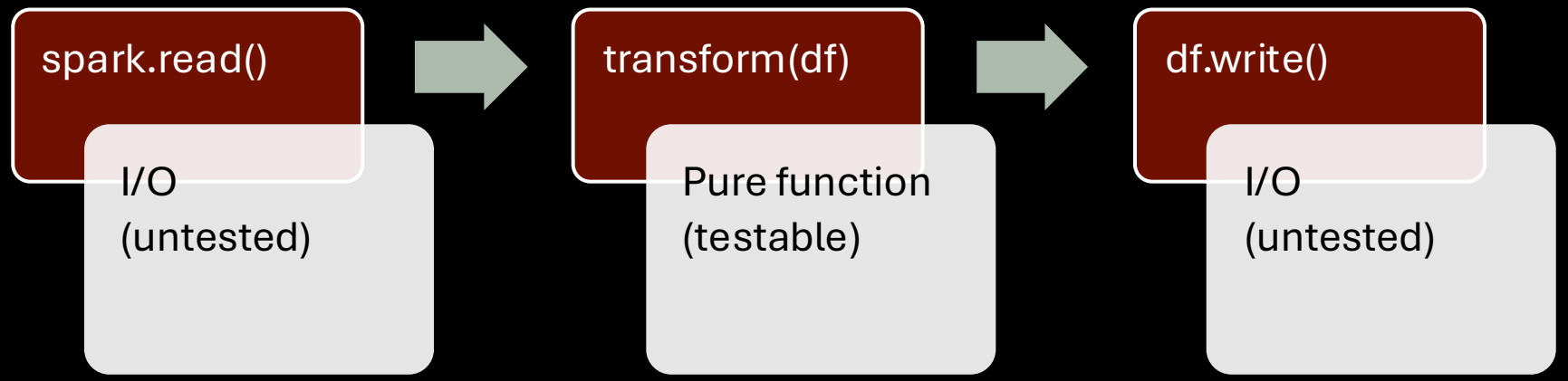
Databricks Connect v2
minutes

pyspark.testing
seconds

pytest
milliseconds

Test at every layer. Start at the bottom.

Separate Logic from I/O



If it reads or writes, it can't be unit tested.

PySpark Test Code

```
def calculate_order_totals(df: DataFrame) -> DataFrame:
    return df.withColumn("total", col("quantity") *
col("unit_price")) \
            .filter(col("total") > 0)
```

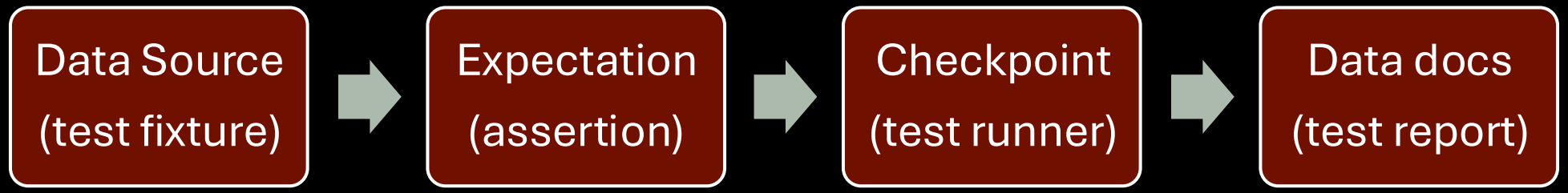
PySpark Test Code

```
from pyspark.testing import assertDataFrameEqual
import pytest

@pytest.fixture(scope="session")
def spark():
    return SparkSession.builder.master("local").appName("tests").getOrCreate()

def test_order_totals(spark):
    input_df  = spark.createDataFrame(
        [(1, 5, 10.0), (2, 0, 10.0)],
        ["order_id", "quantity", "unit_price"]
    )
    expected_df = spark.createDataFrame(
        [(1, 5, 10.0, 50.0)],
        ["order_id", "quantity", "unit_price", "total"]
    )
    result = calculate_order_totals(input_df)
    assertDataFrameEqual(result, expected_df, checkRowOrder=False)
```

Great Expectations Mental Model




```
expect_column_values_to_not_be_null("customer_id")
expect_table_row_count_to_be_between(min_value=1000000, max_value=5000000)
expect_column_values_to_match_regex("email", r"^[^@]+@[^@]+\.[^@]+$")
```

GX is a test suite. For your data.

100+ built-in expectations. Works with Spark, SQL, Pandas.

Dynamic Thresholds

```
expect_table_row_count_to_be_between(min=1_000_000)
```

```
expect_table_row_count_to_be_between(
    min=yesterday_count * 0.90,
    max=yesterday_count * 1.10
)
```

Static assertions aren't enough.

Use Evaluation Parameters - thresholds that learn from your data.

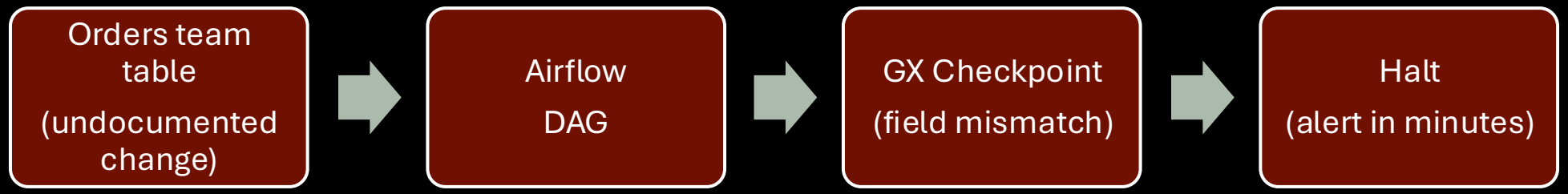
dbt Tests

```yaml
models:
  - name: orders
    columns:
      - name: order_id
        tests:
          - unique
          - not_null
      - name: status
        tests:
          - accepted_values:
              values: ['pending', 'shipped', 'delivered']
              severity: error
```

Your dbt YAML is already a test. Use it.

Same mental model as test categories. Smoke test vs. blocking test.

How GX Caught the Drift for Me



Upstream team said they documented every change.
They missed **two**.
GX caught them instantly.

Streaming: The New Challenges

Event time ≠ Processing time	Which clock governs your aggregations?
Watermarks	How long do you wait for late data?
State	Unbounded growth if you're not careful
Backpressure	Does your system fail gracefully, or just fail?

Kafka Testing Matrix

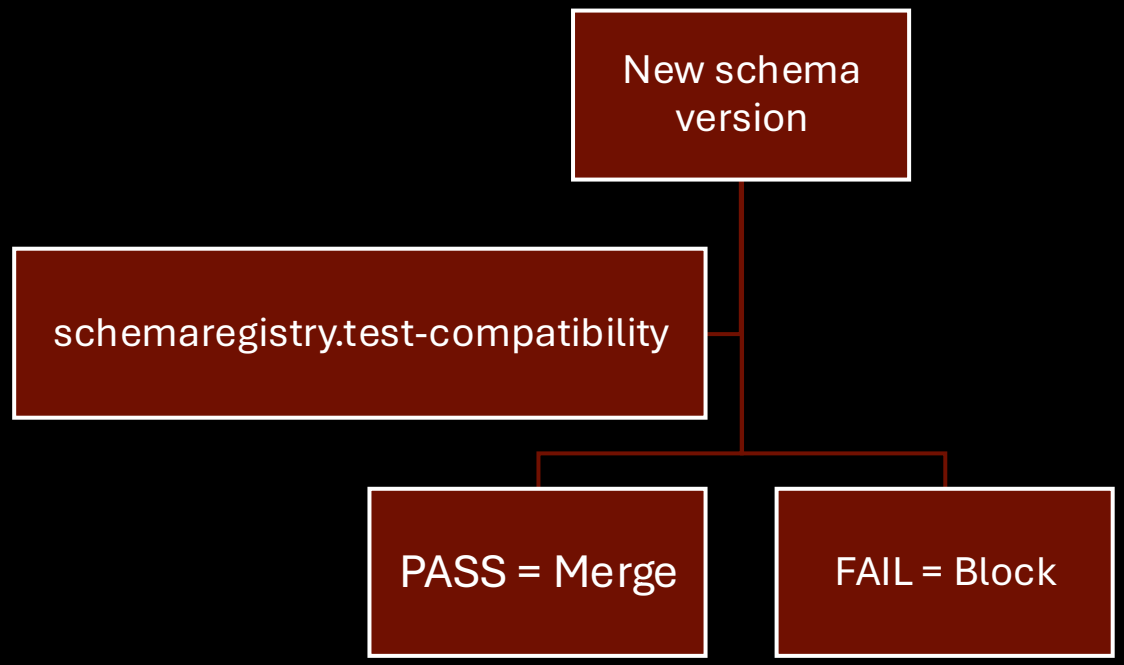
What you're testing	Tool	Why
Serialization / partition logic	MockProducer / MockConsumer	In-process. Milliseconds.
Kafka Streams topology	TopologyTestDriver	Deterministic. Control time.
Real broker behavior	Testcontainers Kafka	Full fidelity. Docker -managed.

Dead Letter Queue Test

What happens when bad data arrives?

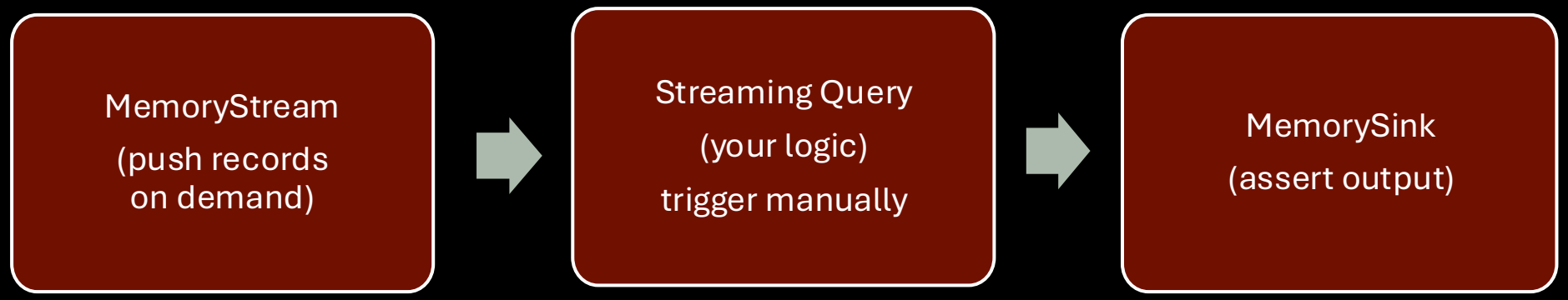
Write a test for this. Most teams don't.

Schema Compatibility CI Gate



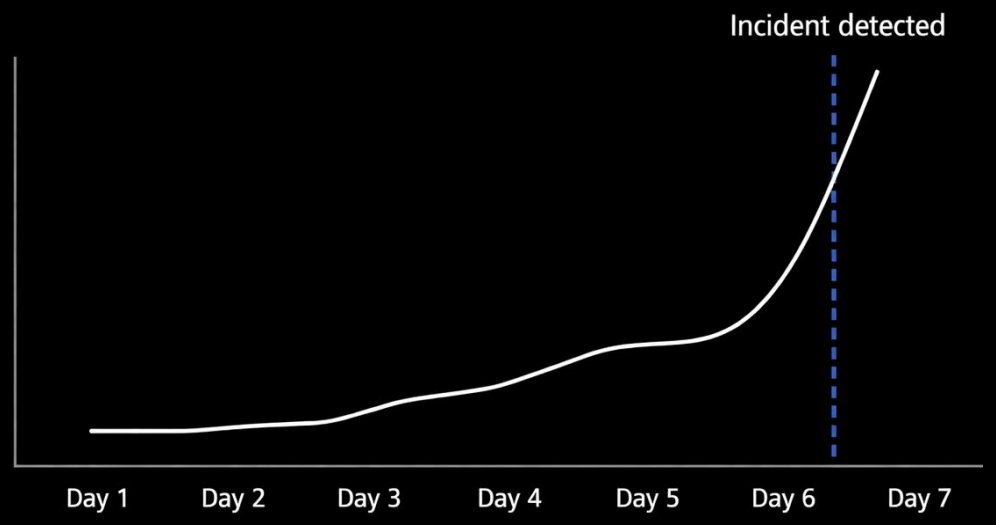
BACKWARD	FORWARD	FULL
New reads old data.	Old reads new data.	Both directions guaranteed.

MemoryStream + MemorySink



Full control over time and data.

A Story about Consumer Lag



Consumer Lag

✗ What we had: monitoring on job success/failure

✓ What we needed: alert when lag > threshold

Trusting your Data

Schema Drift

Synthetic Data

Data Contracts

Three concepts that separate
"we have tests" from "we **trust** our data."

Schema Drift

order_id	INT
customer_id	INT
order_date	DATE
total	FLOAT

Before

Schema Drift

order_id	INT	
customer_id	INT	
order_date	DATE	
amount	FLOAT	renamed
currency	STRING	new

After

Schema Drift Defense Layers

What	When	
GX schema check	at ingestion	catches at landing
Delta Enforcement	default-on	blocks bad writes
DLT RESCUE mode	captures unknown	preserves for replay
Schema Registry	Kafka	prevents at source

Synthetic Data

```
from faker import Faker
from pyspark.sql import functions as F

fake = Faker()
# Generate 10M synthetic user events
df = spark.range(10_000_000).select(
    F.col("id").alias("event_id"),
    F.rand().alias("session_duration"),
    F.when(F.rand() < 0.05, None)
     .otherwise(F.monotonically_increasing_id()).alias("user_id"),
    F.from_unixtime(F.unix_timestamp() -
        (F.rand()*86400)).alias("event_time")
)
```

Synthetic Data

Spark rand()

100M rows, free.
Parallelizes on
your cluster.

→

Faker

Realistic PII.
Names,emails,
addresses, IDs.

→

Gretel.ai

Enterprise fidelity.
Differential privacy.
Compliance-safe.

Engineer your edge cases deliberately –
nulls, boundary values, bad types.

Data Contracts

```yaml
paths:
 /orders:
   get:
     responses:
       200:
         schema:
           $ref: Order
```

OpenAPI (REST API)

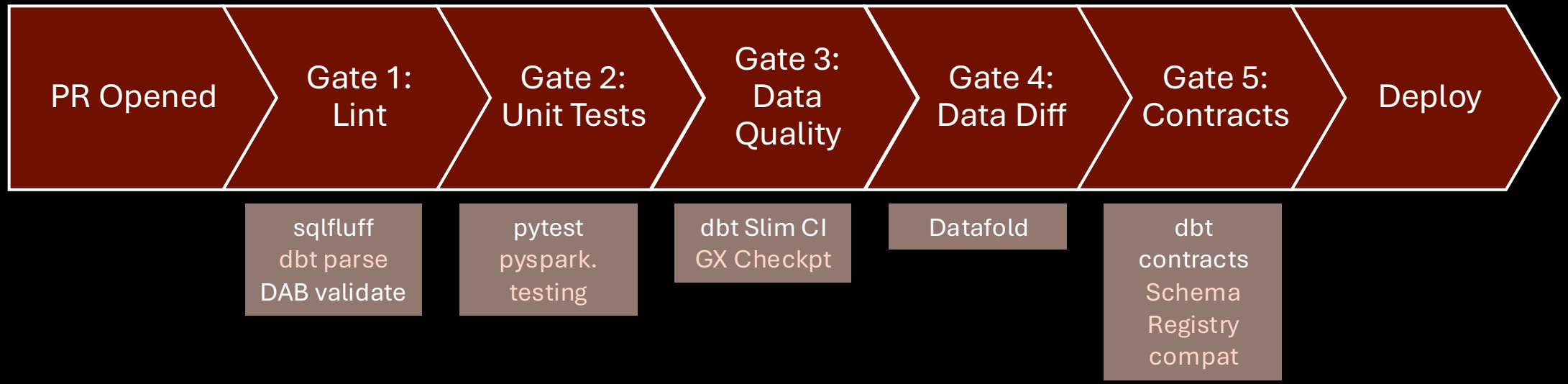
```yaml
dataset: orders
columns:
  - name: order_id
    type: integer
    nullable: false
  freshness_sla: 1h
owner: data-platform
```

ODCS (Data Contract)

A data contract is an API spec. For your pipeline.

Schema, types, nullability, SLAs, ownership. Versioned. In source control. Enforced in CI.

CI/CD Quality Gates



Five gates. Every PR. No exceptions.

Slim CI

Without Slim CI	Run all 400 dbt models	42 minutes
With Slim CI	Run 12 changed models	4 minutes

```
dbt build --select state:modified+
```

10× faster feedback. Same confidence.

"Data is only as trustworthy as the tests behind it."

Start with one test. Ship it this week.